ESN Uncertainty Pipeline Replication Guide

A complete step-by-step guide to replicate the ESN uncertainty analysis and adapt it for any robotic arm manipulation task.

Author	Umar Bin Muzzafar
Organization	OMNIBIO LTD
Date	April 08, 2026
Python	3.10 (required for LeRobot)
GPU Required	No — runs on CPU
Training Time	< 2 seconds

Contents

1. What Is an ESN? (Architecture Explained)

An Echo State Network (ESN) is a type of recurrent neural network. Unlike normal neural networks, the recurrent connections are **random and fixed** — they never get trained. Only a simple linear layer (the readout) is trained. This makes ESNs:

- **Extremely fast** — training is a single matrix solve, not thousands of epochs
- **CPU-only** — no GPU needed, runs in under 2 seconds
- **Stable** — no vanishing gradients, no catastrophic forgetting
- **Good at temporal data** — the reservoir maintains memory of past inputs

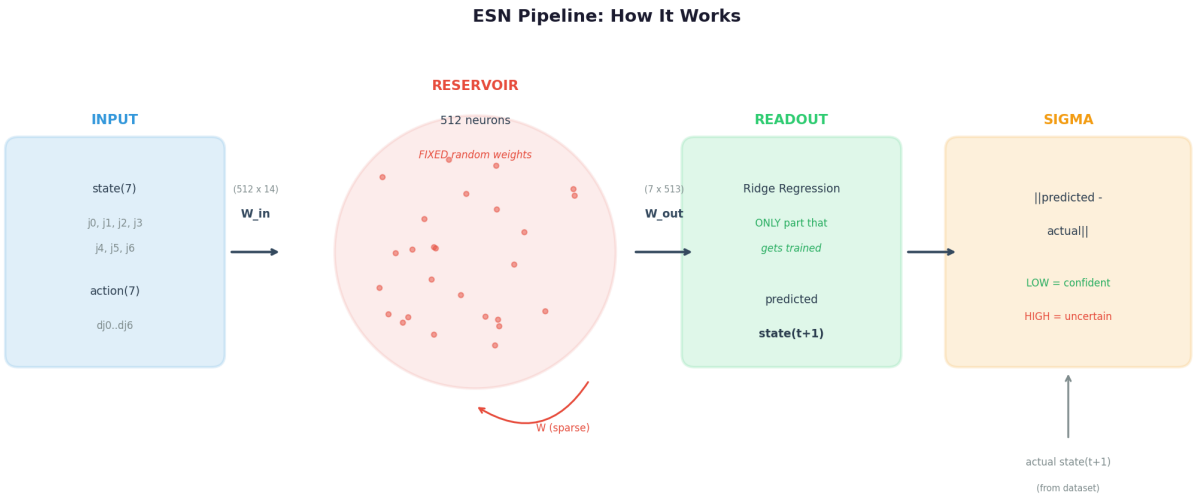

Figure 1: ESN pipeline architecture

How It Works — Step by Step

1. **Input (14 dims)**: At each timestep, we concatenate the robot's current joint state (7 values) + the action command (7 values) = 14 numbers.

2. **W_in projection (512 x 14)**: A random matrix projects the 14-dim input into a 512-dimensional reservoir space.

3. **Reservoir update**: The 512 neurons update their state using: `x(t) = (1-α)·x(t-1) + α·tanh(W_in·u + W·x(t-1))` where W is a sparse random matrix and α is the leaking rate. This equation means each neuron mixes its previous state with new input — creating memory.

4. **W_out prediction (7 x 513)**: A trained linear layer maps the 512-dim reservoir state to a 7-dim predicted next state. The +1 is a bias term.

5. **Sigma**: We compare the prediction to what actually happened: `σ = ||predicted - actual||`. Low σ = familiar. High σ = unexpected = uncertain.

The Spectral Radius

The spectral radius (SR) of W controls how long the reservoir remembers past inputs. SR=0.95 means echoes of past inputs persist for many timesteps — good for predicting robot trajectories. If SR were > 1.0, the network would be unstable (states explode). If SR < 0.5, memory would be too short.

Ridge Regression (Training)

Training the readout is a single equation, not gradient descent:

$$W_{out} = (S^T \cdot S + \alpha \cdot I)^{-1} \cdot S^T \cdot Y$$

S = matrix of reservoir states (one row per timestep)
Y = matrix of actual next states (targets)
α = regularization (prevents overfitting, default 0.01)

This solves instantly. No learning rate, no epochs, no batch size. Just one matrix inversion.

2. Project Structure

Project Structure

```
omni3/
├── __init__.py
├── config.py                    ← All parameters (ESN, thresholds, paths)
├── requirements.txt
│
├── data/
│   ├── loader.py                ← UR5 dataset loader (LeRobot)
│   └── droid_loader.py          ← DROID-100 loader + task filtering
│
├── core/
│   ├── esn.py                   ← Echo State Network (512 neurons, FIXED)
│   ├── readout.py               ← Ridge regression (ONLY trained part)
│   └── uncertainty.py           ← Sigma computation + zones
│
├── analysis/
│   ├── filter.py                ← Anomaly detection (4 types)
│   ├── phase1.py                ← Per-joint, temporal, classification
│   └── task_analysis.py         ← Learning curve + task-specific
│
├── viz/
│   ├── rerun_logger.py          ← Rerun.io visualization
│   └── blueprint.py             ← Rerun viewer layout
│
├── pipeline.py                  ← Orchestrator (train + evaluate)
│
├── scripts/
│   ├── 00_explore.py            ← Step 1: Inspect dataset
│   ├── 01_filter.py             ← Step 2: Find anomalies
│   ├── 02_train_esn.py          ← Step 3: Train ESN
│   ├── 03_validate.py           ← Step 4: Validate sigma
│   ├── 04_visualize.py          ← Step 5: Rerun viewer
│   ├── 06_droid_failures.py     ← Step 6: Success vs failure
│   ├── 07_phase1_analysis.py    ← Step 7: Deep analysis
│   └── 08_task_specific.py      ← Step 8: Task-specific + learning curve
│
└── output/                      ← All generated results
```

Figure 2: Full project file tree with descriptions

Directory	Purpose
config.py	All parameters: ESN size, thresholds, paths. Change these first.
data/	Dataset loading. loader.py for UR5, droid_loader.py for DROID-100.
core/	The brain: esn.py (reservoir), readout.py (ridge regression), uncertainty.py (sigma)
analysis/	Anomaly detection + Phase 1 analyses + task-specific learning curve
viz/	Rerun.io visualization (interactive trajectory + sigma timeline viewer)
scripts/	Numbered scripts you run in order. Each one does one job.
pipeline.py	Wires core/ together: train on episodes, evaluate sigma.
output/	All generated results: models, plots, JSON, Rerun recordings.

3. Prerequisites & Setup

System Requirements

Requirement	Details
Python	3.10 (required by LeRobot). Check: python3.10 --version
OS	Linux (tested on Ubuntu 22.04). macOS should work.
RAM	4 GB minimum (dataset is ~100MB)
GPU	NOT required. Everything runs on CPU.
Disk	~2 GB for dataset cache + outputs

Installation

```
# 1. Clone the repository
git clone https://github.com/OMNIBIO-LTD/Active-inference.git
cd Active-inference
git checkout Umz_Work

# 2. Install dependencies
pip install -r omni3/requirements.txt

# 3. Verify installation
python3.10 -c "from lerobot.datasets.lerobot_dataset import LeRobotDataset;
print('OK')"
python3.10 -c "import rerun; print('Rerun', rerun.__version__)"
python3.10 -c "import scipy; print('SciPy', scipy.__version__)"
```

The dataset downloads automatically from HuggingFace on first run. No manual download needed. It's cached in ~/.cache/huggingface/ after the first run.

4. Step-by-Step: Run the Full Pipeline

Run these commands from the repository root (where omni3/ directory is). Each script is standalone and produces output in omni3/output/.

Step 0: Explore the Dataset

See what's in the UR5 dataset — episodes, tasks, state/action dimensions.

```
python3.10 omni3/scripts/00_explore.py

# For only first 20 episodes (faster):
python3.10 omni3/scripts/00_explore.py --episodes 20
```

This prints: number of episodes, frame lengths, state/action ranges, task list, success rate. Takes ~7 seconds for 20 episodes.

Step 1: Detect Anomalies

```
python3.10 omni3/scripts/01_filter.py --episodes 100
```

Scans 100 episodes for velocity spikes, acceleration spikes, command gaps, and gripper oscillation. Outputs: omni3/output/filtered_episodes.json

Step 2: Train ESN

```
python3.10 omni3/scripts/02_train_esn.py --train-episodes 100
```

Trains the ESN readout on 100 episodes using ridge regression. Takes < 1 second. Saves model to omni3/output/models/esn_readout.npz. Expected RMSE: ~0.025 for UR5.

Step 3: Validate Sigma

```
python3.10 omni3/scripts/03_validate.py
```

Evaluates sigma on flagged episodes. Checks whether sigma spikes before anomalies.

Step 4: Visualize in Rerun

```
# Save to file (can share with others):
python3.10 omni3/scripts/04_visualize.py --episodes 0 1 2 --save-rrd
omni3/output/viz.rrd

# View the recording:
rerun omni3/output/viz.rrd
```

Opens an interactive Rerun viewer with: EE trajectory (3D), state time series, sigma timeline, anomaly markers, and per-dimension prediction error.

Step 5: Success vs Failure (DROID-100)

```
python3.10 omni3/scripts/06_droid_failures.py --save-rrd omni3/output/droid.rrd
```

Loads DROID-100 (Franka Panda, 81 success + 19 failures). Trains ESN on successes, measures sigma on failures. Compares sigma between groups.

Step 6: Deep Analysis (Phase 1)

```
python3.10 omni3/scripts/07_phase1_analysis.py
```

Three analyses: (1) per-joint sigma — which joints predict failure, (2) temporal profile — when does sigma diverge, (3) ROC/AUC classification. Plots saved to omni3/output/phase1/.

Step 7: Task-Specific + Learning Curve

```
python3.10 omni3/scripts/08_task_specific.py
```

The most important script. Focuses on one task (Task 5, 35 success + 19 failure). Runs the incremental learning curve: trains on 1, 2, ..., 30 episodes and shows sigma decreasing. Proves the ESN builds a generative model. Plots saved to omni3/output/task_specific/.

5. How to Adapt for a New Dataset / Task

The pipeline works with **any LeRobot-format dataset**. Here's how to plug in a new one:

Step A: Find a Dataset

Browse HuggingFace for LeRobot datasets: https://huggingface.co/datasets?search=lerobot

Dataset	Robot	Episodes	Has Failures?
lerobot/berkeley_autolab_ur5	UR5	1,000	No
lerobot/droid_100	Franka	100	Yes (19)
lerobot/berkeley_fanuc_manipulation	Fanuc	415	Unknown
lerobot/roboturk	Franka	1,995	Unknown
lerobot/stanford_kuka_multimodal_dataset	KUKA	3,000	Unknown
IPEC-COMMUNITY/kuka_lerobot	KUKA IIWA	209,880	Unknown

Step B: Inspect the Dataset

```
python3.10 -c "
from lerobot.datasets.lerobot_dataset import LeRobotDataset
ds = LeRobotDataset('YOUR_REPO_ID', episodes=[0], download_videos=False)
hf = ds.hf_dataset
sample = hf[0]
print('Features:', list(ds.features.keys()))
print('State shape:', sample['observation.state'].shape)
print('Action shape:', sample['action'].shape)
print('Episodes:', ds.num_episodes)
"
```

Step C: Update config.py

Edit `omni3/config.py` with your dataset's parameters:

```
DATASET_REPO_ID = 'your/dataset_id'
DATASET_FPS = 15 # your dataset's FPS

STATE_DIM = 7 # your observation.state dimension
ACTION_DIM = 7 # your action dimension
ESN_INPUT_DIM = STATE_DIM + ACTION_DIM # auto-computed
READOUT_OUTPUT_DIM = STATE_DIM # predict next state

STATE_NAMES = ['j0', 'j1', ...] # your joint names
ACTION_NAMES = ['a0', 'a1', ...] # your action names
```

Step D: Write a Loader (if needed)

If your dataset has the same structure as UR5 (observation.state, action, next.reward, next.done), scripts 00-04 work directly — just change config.py. If the field names differ, create a new loader in data/ following the pattern in droid_loader.py. The key is to output a list of dicts with:

```
episode = {
'index': int,
'states': np.ndarray, # (T, STATE_DIM)
'actions': np.ndarray, # (T, ACTION_DIM)
'timestamps': np.ndarray, # (T,)
'success': bool,
'task': str,
'num_frames': int,
}
```

Step E: Run the Pipeline

That's it. Run scripts 00-08 in order. The ESN, readout, uncertainty, and analysis code are dataset-agnostic — they work with any (state, action) pair of any dimension.

> **The ESN_INPUT_DIM and READOUT_OUTPUT_DIM in config.py are the only things you must change** to use a different robot. Everything else adapts automatically.

6. Understanding the Output

The Learning Curve (Most Important)

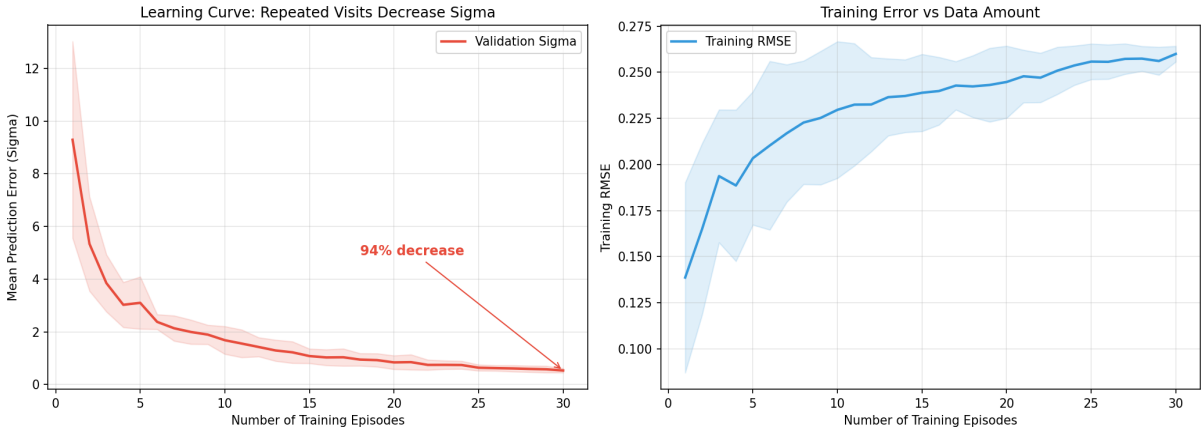

Figure 3: Validation sigma drops 94% as training data grows (left). Training RMSE stabilizes after ~15 episodes (right).

Left plot: X-axis = number of training episodes. Y-axis = prediction error on held-out validation episodes. The curve should go DOWN — this proves the ESN learns from repeated visits. A flat curve means the ESN isn't learning.

Right plot: Training RMSE. Goes UP because more data means more variety, but the model generalizes better (validation sigma goes down).

Per-Joint Comparison

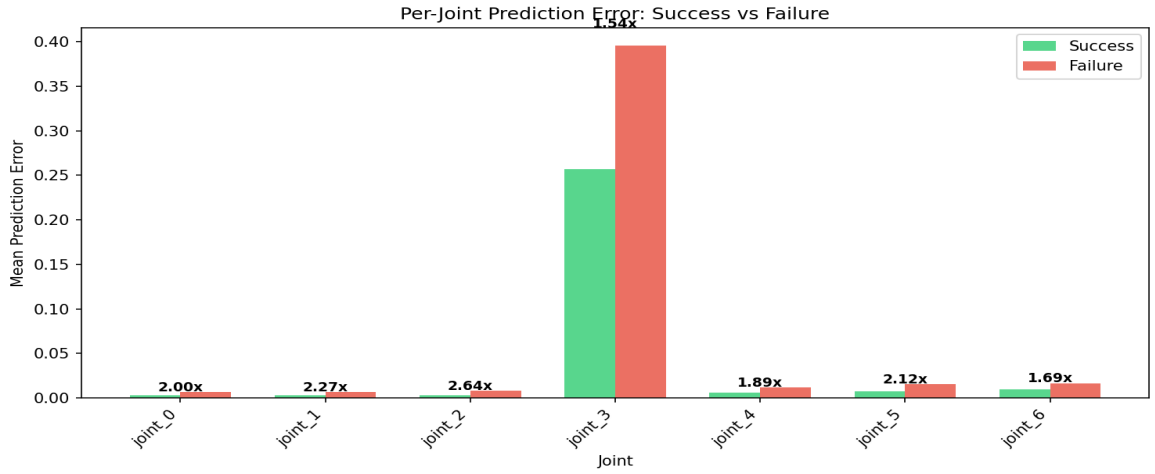

Figure 4: Per-joint prediction error. Numbers above bars = failure/success ratio.

Green bars = average prediction error for successful episodes. Red bars = average for failed episodes. The ratio (number above) tells you which joint is most predictive of failure. Higher ratio = that joint deviates more during failures.

ROC Curve

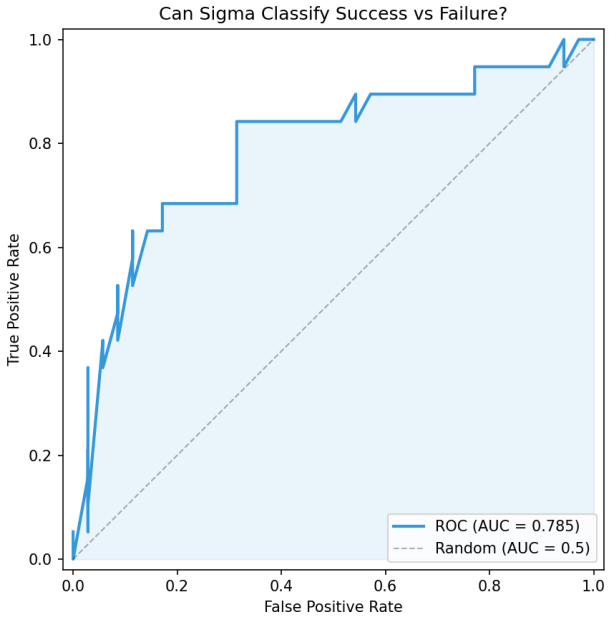

Figure 5: ROC curve for sigma-based failure classification.

AUC (Area Under Curve): 0.5 = random, 0.7 = moderate, 0.8 = strong, 1.0 = perfect. Our task-specific AUC is 0.785 using a single feature (sigma_mean). The curve shows the tradeoff between catching failures (recall) and false alarms (FPR).

7. Key Parameters to Tune

Parameter	Default	What It Does	When to Change
RESERVOIR_SIZE	512	Number of neurons. More = more capacity	Increase for complex tasks (1024, 2048)
SPECTRAL_RADIUS	0.95	Memory length. Higher = longer memory	Lower (0.8) for fast tasks, higher (0.99) for slow
LEAKING_RATE	0.3	How fast old state decays. 0=keep all, 1=forget all	Lower (0.1) for slow robots, higher (0.5) for fast
INPUT_SCALING	0.3	Scale of input projection.	Increase if inputs have large range
RIDGE_ALPHA	0.01	Regularization. Higher = less overfitting	Increase if RMSE is good but sigma is noisy
ESN_WARMUP_STEPS	10	Discard first N frames (transient)	Increase for slower FPS datasets
SIGMA_WINDOW_SIZE	50	Running average window for z-score	Decrease for shorter episodes

Start with defaults. Only tune if results are poor. The most impactful parameter is RESERVOIR_SIZE — try 1024 if 512 gives weak sigma separation.

8. What We Found (Summary)

Finding	Result	Significance
Learning curve	94% sigma decrease over 30 episodes	ESN builds a generative model
Sigma ratio (F/S)	1.54x (task-specific)	Failures are distinguishable
Classification AUC	0.785	Moderate classifier from 1 feature
Most predictive joint	joint_2 (2.64x)	Upper arm fails, not wrist
Early warning	27% completion	73% warning time remaining
Training time	< 1 second	Real-time capable

The ESN uncertainty pipeline is validated: it learns task dynamics, detects failures early, and runs in real-time on CPU. Next steps are BCM weight adaptation and Curious/Cautious Pi policies for active robot behavior under uncertainty.

Quick Start (TL;DR): Install deps, then run `python3.10 omni3/scripts/08_task_specific.py` to see everything.